Ditto: A DAG Visualization Tool for Pyro Probabilistic Models
By Sam Nelson and Sonya Rashkovan

Introduction

Reinforcement learning (RL) is a powerful framework for training agents to act in complex environments, but designing the reward functions that guide those agents is notoriously difficult. Even experienced practitioners must make probabilistic assumptions at every step: Which features does the agent actually depend on? Will the rewards I specify produce the behavior I intend? What prior beliefs am I encoding, and how much will observed outcomes shift them? These questions are fundamentally Bayesian, and getting them right is the difference between an agent that converges on a useful policy and one that exploits unintended loopholes.

In practice, the probabilistic reasoning underlying RL reward design is largely invisible. A practitioner writing a Pyro model encodes distributions over parameters and observations in Python source code, then waits for training runs to complete before inspecting results. If the behavior is unexpected, they must reason backward from outputs to figure out where their probabilistic assumptions went wrong. This iteration loop is slow, cognitively taxing, and requires a rare combination of RL expertise and Bayesian fluency.

We developed Ditto to address these pain points. Ditto is a static analysis and inference tool that parses annotated Pyro model files, constructs a directed acyclic graph (DAG) of variable dependencies, runs Stochastic Variational Inference (SVI) to approximate posterior distributions, and renders an interactive web application where practitioners can hover over any variable in the DAG to see both its prior and posterior distributions side by side. The key insight is that making the probabilistic structure of a model visible, and showing how inference shifts distributions away from their priors, dramatically reduces the cognitive overhead of reward design.

Experts in our lab, GIRAFFE, identified three recurring pain points that motivate the tool. First, re-running models and then doing post-hoc statistical analysis on changes in distributions is time-consuming; a change to one line of a model can require an entire new training run before the distributional consequences become visible. Second, the magnitude, direction, and efficiency of distributional change based on code changes are often not intuitive or guessable, especially in models with complex hierarchical dependencies. Third, not every RL expert is a statistics expert, meaning that succeeding at both tasks requires developing two separate skill sets simultaneously.

The closest existing tool is Dagitty, a browser-based DAG editor aimed at causal inference researchers who use R. Dagitty is useful for reasoning about causal structure but has several limitations that make it unsuitable for the RL reward design context: it is R-based, making it inaccessible to practitioners working in PyTorch/Pyro; it is primarily designed for causal inference rather than probabilistic programming; and after over a decade of active development, it still does not run inference or visualize distributions from live models. Matplotlib, the default tool for ad hoc visualization in Python, requires the user to manually extract samples, write plotting code, and re-execute it after every model change. It has no concept of model structure, no DAG view, and no integration with the Pyro inference pipeline.

This project was also grounded in principles from the Tools for Thought course. Ditto applies the externalization principle: by rendering distributions as persistent visual artifacts alongside the model structure, the tool reduces the working memory load on the practitioner. It applies selective attention design by drawing the eye to the difference between prior and posterior, the signal that is most diagnostic for understanding whether inference is working as intended. More broadly, the tool treats the cognitive experience of the RL modeler as a first-class design constraint alongside correctness.

Approach

Ditto is a command-line Python tool that takes a Pyro model file as input and produces an interactive browser-based visualization as output. A user points Ditto at their model file, and within

seconds sees a DAG of their model's variables, with distribution plots available on hover for every annotated node. The architecture consists of five pipeline stages: annotation parsing, dependency graph construction, inference, visualization, and a CLI orchestration layer.

User Story: Designing a Reward Function with Ditto

Suppose a practitioner is designing a Pyro model to study whether Ditto cheated on the TfT midterm exam. The user would create a model using relevant data (e.g., Ditto's score, time taken, Knowledge, etc) along with constraining assumptions (e.g., Stress is the driving factor behind performance and time taken). Assuming there is a closed form (we do so with a fabricated version later), the user could use the observed score and time taken to calculate the most likely value for 'cheated' to explain the data. This process relies on numerous assumptions that propagate through each variable. The DAG provides an overview of how those values propagate and the density plots and histograms show the estimated distributions allowing the user to piece together the logical flow of the model. Once the user has created the model, all they have to do is mark the relevant data with the comment **# !Ditto: <tags>** (immediately above the variable).

Compare this to the Matplotlib baseline. To produce the same comparison, the practitioner would need to: (1) write an AutoNormal or custom guide for each latent variable, (2) run SVI manually and track the param store, (3) instantiate Predictive, (4) call it with the trained guide, (5) extract the samples dict, (6) write plt.figure() and ax.plot() calls for prior draws, (7) write the same for posterior draws, (8) call plt.show(), and (9) repeat for every variable of interest. Ditto compresses this workflow to one annotation per variable and one CLI command (for an existing model).

Why the DAG?

The DAG representation is central to the design, not merely decorative. Reward design in RL relies heavily on understanding what features the agent depends on to make decisions. The dependency structure of a Pyro model encodes exactly this: which random variables influence which others, and through what pathways. Key diagnostic questions become answerable by inspecting the graph topology: Which features are important to the agent? What features does the reward designer believe will elicit the proper policy? Are there unintended dependencies that could produce reward hacking?

This line of questioning is fundamentally Bayesian. Reward designers are trying to answer the question: what is the likelihood of the desired behavior given these specific reward weights, and how can we maximize that likelihood by adjusting the weights? The DAG makes the prior assumptions and their downstream consequences legible. It also grounds the distribution visualizations in context: knowing that the posterior for variable A shifts dramatically only matters if you can see that A is a parent of B and C in the reward structure.

Architecture and Implementation

The pipeline consists of five modules. The parser (parser.py) uses a two-pass strategy: a regex pre-scan identifies # !Ditto: comment lines (necessary because Python's AST module does not include comment nodes), and an AST walk associates each annotation with the variable declaration immediately following it. Tags may be comma-separated (e.g., # !Ditto: prior, latent) to indicate that a variable should display both its prior and its posterior. The parser handles both simple assignment statements and bare pyro.sample(...) expression statements inside model functions.

The graph builder (graph_builder.py) infers directed edges by extracting variable name references from each annotated variable's right-hand-side expression using the AST. A key design challenge was handling unannotated intermediate variables: if stress depends on mu_k (unannotated) which depends on knowledge (annotated), we want a direct edge stress to knowledge. The graph builder resolves these

transitive dependencies by collecting all assignments in the source file and walking the reference graph recursively.

The inference engine (inference_engine.py) handles three cases. For prior and latent variables with bare distribution expressions (e.g., dist.Beta(2.0, 2.0)), it evaluates the expression and draws samples directly. For latent variables declared with pyro.sample(...) calls inside a model function, it runs a prior predictive pass using Pyro's Predictive class, since these expressions cannot be evaluated in isolation. When any latent variables are present, it automatically constructs a variational guide (AutoNormal for simple models, AutoNormalizingFlow for models with complex latent geometry), runs SVI, and collects posterior samples. Critically, it uses poutine.uncondition to strip observation conditioning before posterior predictive sampling, which is more robust than manually removing observed kwargs.

The visualizer (visualizer.py) renders KDE plots using scipy.stats.gaussian_kde with Silverman bandwidth, falling back to histograms for discrete or low-variance variables. Critically, all distribution plots are pre-computed as base64-encoded PNG images at app startup, so the hover callback requires only a dictionary lookup. This keeps hover latency near zero regardless of model size. The Dash + Cytoscape web app uses a flex-row layout with the DAG on the left (three-quarters width) and a side panel on the right that updates on node hover.

An important design decision was eliminating the need for a user-declared guide variable. Earlier prototypes required a # !Ditto: approx annotation, which placed an extra burden on the user and was redundant: the guide choice (AutoNormal vs. AutoNormalizingFlow) can be made automatically by inspecting whether any latent variables are declared via pyro.sample(...) calls. Removing this annotation simplified the user-facing API substantially.

Distribution Tag Design

Ditto uses three tags, each with a distinct visual representation informed by the type of uncertainty it represents:

Observed (orange): Observed variables are variables for which we can take samples. We plot the raw values and fill in gaps using posterior predictive calculations made after the latent variables have been approximated using SVI. We plot an approximated distribution using kernel density estimates to create a smooth curve.

Prior (blue): Assigning priors is one of the great strengths and great risks with bayesian statistics. The ability to constrain posterior distributions allows for higher performance inference when there is limited data but also relies on assumptions made about the validity of the prior. Variables tagged “# !Ditto: prior” are visualized according to the distribution provided in the variable below. For example, “pyro.sample(“infection_perc”, dist.Beta(1.0,1.0))” would be shown as a Beta distribution with alpha=1.0 and beta=1.0.

Latent (green): Latent variables are variables that are not observed (these often include priors) but typically have a justified shape. For example, assuming there is a normal prior N ~ Norm(5, 22), a gamma latent G ~ Gam(N, 10), and a normal observed O ~ Norm(G*2, 2), the latent variable may not be observed but its parameters may be inferred using baye’s rule. We can create a distribution over the likely values of G based on our observed samples from O.

Combined prior/latent (blue/green): Prior variables are always latent but not always a target for inference. For priors that are also targets for inference, we allow both tags. We then visualize both the prior distribution and its posterior allowing the user to compare the two.

Evaluation

We evaluated Ditto through two case studies drawn from real models used in our lab. Because Ditto is a developer tool, we focus on whether it reduces the manual work required to reach the same distributional insights that Matplotlib-based workflows produce, and whether it surfaces insights that the Matplotlib baseline would miss entirely.

Ditto is cheating

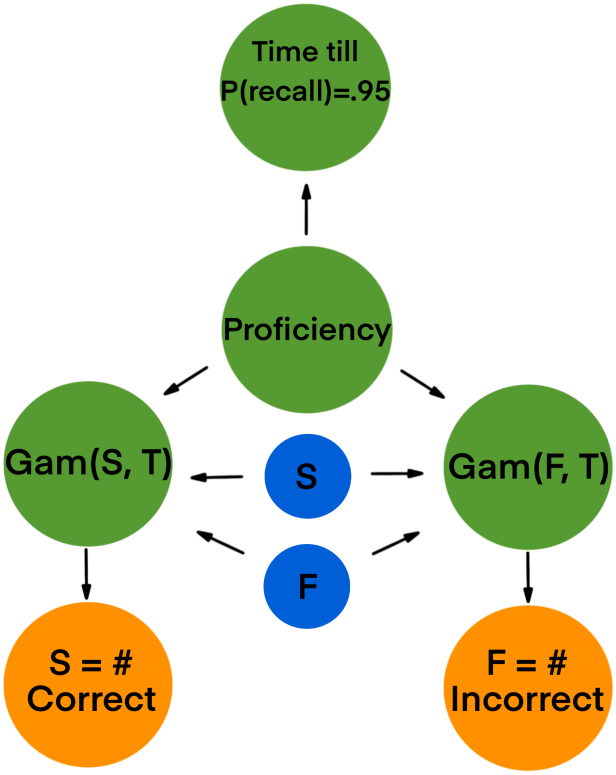
The motivating example for the tool came from a classroom scenario: our dear Ditto scores unusually high on the TfT exam, and there is a question of whether this reflects genuine performance or cheating. We modeled this as a Bayesian inference problem with latent variables for true ability and a prior probability of cheating. The DAG below shows our model as well as the prior and posterior distributions for Ditto's stress:

In the Matplotlib baseline, this would require extracting prior and posterior samples separately, writing histogram or KDE plotting code for each, visually aligning the plots, and re-running whenever the model changes. In Ditto, the practitioner adds annotations to the relevant variables and runs the CLI or loads the file directly through the GUI. In our model, with the observed score data, the posterior shifted toward lower cheating probability, exonerating Ditto. This result was visible upon hovering over the node—the only requirement was for the user to tag the relevant variable and upload the file.

The DAG view added a dimension the Matplotlib baseline cannot provide: it made visible that the observed score variable was downstream of both the ability latent and the cheating prior. This dependency structure clarified why changing the prior on cheating had a downstream effect on the score likelihood that was not intuitive from looking at the code alone. A practitioner using only Matplotlib would have to mentally reconstruct this dependency structure each time; Ditto displays it automatically.

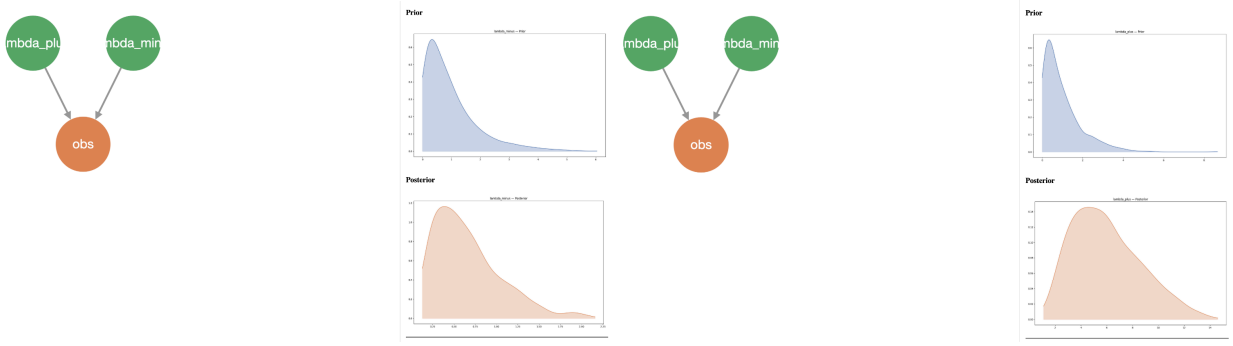
SRS Example

We also implemented a spaced repetition algorithm based on competing risks and attempted to use our tool to illustrate how it works. We include this example as an illustration of how our model fails with complex systems that require custom distributions in Pyro. This spaced repetition system was inspired by competing risks models in which two related rates (i.e., successful and unsuccessful recalls) "compete" to see which arrives first. There are several reasons why this is not an optimal way to design an SRS; as that is incidental to demonstrating our tool's (in)capabilities we will not spend an extended amount of time on it. Below is the DAG for the SRS:



The changing successes and failures (S and F respectively) are observed values that contribute to the estimate of the student's estimated proficiency with a given notecard. In the DAG above, the successes and failures within a specified time (left and right Gamma distributions respectively) are determined by the student's Proficiency with the notecard. The Proficiency is modeled based on 1-hazard growth calculated using the expected value of the Gamma distributions. This allows us to calculate the point at which the probability of failure before success is equal to 0.05 (i.e., when the probability of recall is 0.95 and we need to schedule a review). The priors here represent our initial beliefs about the expected number of successes and failures over a specified unit of time.

Below is the DAG for our srs_example.py (with information for the gamma distributions, Successes Gamma on the right, Failures Gamma on the left):



Evidently, this looks significantly different from the DAG we displayed above. If we look at the code, we see why displaying the lambdas may be easy, but displaying the remainder is substantially more difficult.

```python
# Model
def model(events):

    # Priors
    # !Ditto: prior, latent
    lambda_plus = pyro.sample("lambda_plus",   dist.Gamma(1.0, 1.0))
    # !Ditto: prior, latent
    lambda_minus = pyro.sample("lambda_minus", dist.Gamma(1.0, 1.0))

    # encode data
    data = torch.tensor([[dt, et] for et, dt in events])

    with pyro.plate("observations", len(events)):
        # !Ditto: observed
        pyro.sample(
            "obs",
            InterEventDistribution(lambda_plus, lambda_minus),
            obs=data
        )
```

While **lambda_plus** and **lambda_minus** are easily distinguishable for the user, the several variables (i.e., the true estimates for S and F) are calculated in **InterEventDistribution**. This is a custom distribution necessary since Pyro does not support a closed form and so must rely on log-likelihood to estimate the distributions of S and F. In order to represent "obs" the custom distribution required an override of Pyro's sample function, an addition that was not required for simple inference. Furthermore, calculating the recall probability depends on the posterior distributions of S and F yet automatically determining the dependency in order to generate an accurate graph is something that eluded us.

Despite the imperfection of the tool for this use case, observing the **lambda_plus**, **lambda_minus**, and **obs** distributions does still offer insight. **lambda_plus** has an obviously higher mode than **lambda_minus** (indicating the user's higher proficiency with the notecard) and **obs**—which is somewhat redundant—indicates that successes are more common (indicating a higher proficiency).

Evaluation Summary - Ultimately our tool fell short of what we hoped it would become—more on this in the limitations section. The Ditto example above shows that when a statistical model can be written within a single model/guide with only Pyro functions it can be useful as a model summary and diagnostic tool. The SRS example reveals that models with custom distributions require additional overhead that makes the tool cumbersome and unwieldy.
We did not include RL applications in our case studies as we could not find any that were readily available. Creating an RL agent simply for the purpose of testing reward design would have involved superfluous complexity. We instead opted to test our tool on select statistical models aiming at complexity as a benchmark for our tool's performance in RL.

Limitations

Our ambitions definitely outstripped our capacity. Our architecture lacked the capability to update in real-time (currently it generates static pngs). This functionality would be particularly useful in reward design and reinforcement learning with human preferences. Being able to adjust reward weights and observe how that affected the agent's model of the designer's desired reward function would be much easier with such functionality.

Our distribution visualizations were sub-par. Similar variables (e.g. lambda_plus and lambda_minus) should have the same axes to make comparison easier. We were not able to implement

this in time, but it does significantly reduce the utility of our tool. For example, the lambda_plus posterior distribution has a higher mode than the lambda_minus posterior, correctly indicating that the observed data indicated that successful recall attempts were more likely than unsuccessful ones. Determining this requires the user to switch between nodes and compare the x-axes.

The biggest obstacles to our tool are the overhead required to produce model code capable of being interpreted by the tool and the inability to interpret more complex models like the SRS example. Variables (and their relationships) rely on explicit pyro API calls. This is necessary in order to track variables in Pyro's computational graph of random variables.

Finally, we were unable to test our tool in a reward design environment. We prioritized more complex statistical models than the ones in IRD to avoid the additional set-up required for an RL agent that would be irrelevant to our project. IRD does use simple linear combinations for the reward functions so we believe that our tool should perform well in this particular use-case. The distributions used in IRD are either the softmax distribution,the normal distribution or linear combinations of them. Our tool should be useful for simple RL environments (as in IRD) but we have not explicitly tested this.

Discussion

Considering that this project was developed by prompting of our lab members, the use of the product is pretty narrow. If we were starting over and attempted to develop a more general tool, the main thing we would do differently is write the user story earlier. The professor's feedback asking for a complete end-to-end user story was exactly right: writing that story forced us to articulate precisely where Ditto improves on the Matplotlib baseline and why each design decision serves the practitioner's actual workflow. We should have done this at the design stage rather than at the writeup stage.

Yet for the tool we developed, the most important next step is user testing with actual lab members. As it stands, simple additive models for reward functions (such as what is used in IRD) may be expressible with the tool. Since we did not have a current RL environment available, we were unable to test its effectiveness in that realm. We believe that our tool should be more than capable of representing the simple reward design environment specified in IRD. With some development, we hope to make it more applicable in the future.

Contribution

Both:

- Interviewed our lab members and Prof Booth
- Presented the project in class

Sam:

- Wrote the outline for the write up and the SRS example
- Developed the second (final) prototype
- Developed the examples

Sonya:

- Brainstormed and designed the UI
- Developed the first prototype
- Prepared the slides for the presentation
- Wrote out the write up